

Tuto pas à pas — Pre-commit, secrets, DVC & MLflow

Repo de départ : thomasfesq/CISIA_29062026 · **Module CISIA** : 24 (CI/CD, tests & versioning) ·

Compétences : C6 (implémenter), C8 (mesurer/maintenir)

Ce tuto part de **l'état réel du repo** et ajoute, étape par étape : les tests + le blocage des secrets en pre-commit, l'installation de **DVC** et **MLflow**, puis deux scripts (scripts/) pour faire une **démo de versioning données + modèle**.

Toutes les commandes utilisent `uv` (déjà en place dans le repo). Sous Windows, ouvre **PowerShell** dans le dossier du repo ; sous macOS/Linux, le terminal.

0. État des lieux (ce que tu as déjà)

Élément	Présent ?	Détail
Python / gestionnaire	oui	3.13, uv, package indusense (src layout)
Gold dataset	oui	data/gold/gold_dataset.csv — 1896 lignes, 4 machines (MACH-0104), pas horaire, panne ≈ 10,5 %
Modèle	oui	artifacts/models/rf.joblib + model_metadata.json (RandomForest, 12 features)
Tests	oui	tests/ (loaders, temporal, package)
CI	oui	.github/workflows/ci.yml (ruff + black + pytest sur 3.13)
Pre-commit (config)	oui	.pre-commit-config.yaml : ruff , black , gitleaks déjà déclarés
Pre-commit (activé)	non	le hook git n'est pas installé on le fait en §1
DVC / MLflow	non	absents des dépendances on les ajoute en §3
Générateur de données	non	créé : scripts/generate_synthetic_gold.py
Démo versioning	non	créé : scripts/demo_versioning.py

ASTUCE — Les deux scripts sont à copier dans le dossier `scripts/` de ton repo (ils sont fournis dans `module24_cicd_versioning/scripts/`).

1. Activer les tests & la qualité en pre-commit

L'idée : à **chaque commit**, git lance automatiquement les vérifications. Si l'une échoue, **le commit est bloqué** on ne pousse jamais de code cassé ou mal formaté.

1.1 Installer les outils dev + le hook git

```
uv sync --extra dev # installe pytest, ruff, black, pre-commit
uv run pre-commit install --hook-type pre-commit --hook-type pre-push
```

La 2^e commande écrit le hook dans `.git/hooks/`. Désormais `git commit` (et `git push`) déclenchent pre-commit.

1.2 Les 3 hooks déjà présents

Ton `.pre-commit-config.yaml` contient :

- **ruff** (`--fix`) : linter ultra-rapide, corrige automatiquement l'importable.
- **black** : formate le code (style unique, fini les débats d'indentation).
- **gitleaks** : **scanne les secrets** (clés API, tokens, mots de passe) — détaillé en §2.

1.3 Ajouter les tests dans pre-commit

On veut faire tourner **pytest**. La suite complète peut être lente : on la met **au push** (pre-push), et on garde le commit rapide. Ajoute ce bloc à la fin de `.pre-commit-config.yaml` :

```
- repo: local
  hooks:
    - id: pytest
      name: pytest (suite complète, au push)
      entry: uv run pytest -q
      language: system
      pass_filenames: false
      stages: [pre-push]
```

Variante « tests au commit » : remplace `stages: [pre-push]` par `stages: [pre-commit]`. Sur une grosse suite, préfère un sous-ensemble rapide, ex. `entry: uv run pytest -q -m "not slow"`.

1.4 Premier passage (sur tout le repo)

```
uv run pre-commit run --all-files
```

Sortie typique : chaque hook affiche Passed, Failed ou Fixed. Si ruff/black corrigent des fichiers, ils apparaissent modifiés tu les `git add` puis recommit. **C'est normal au premier run.**

Mettre à jour les versions des hooks de temps en temps : `uv run pre-commit autoupdate`.

2. Bloquer les secrets (gitleaks)

Pourquoi : une clé API committée par erreur reste dans l'historique git **pour toujours** (même supprimée au commit suivant). gitleaks l'attrape **avant** le commit.

2.1 Démo « le secret est bloqué » (à montrer en séance)

Crée un fichier avec un faux secret, puis tente de le commiter :

```
printf -- '-----BEGIN PRIVATE KEY-----\nMIIBVAIBADANBgkqhkiG9w0BAQEFAASCATEwgg... (faux)\n-----END PRIVATE KEY-----' > fuite_demo.txt
git add fuite_demo.txt
git commit -m "test secret"
```

Résultat attendu — le commit **échoue** :

```
gitleaks.....Failed
- hook id: gitleaks
- exit code: 1

Finding:      -----BEGIN PRIVATE KEY-----...
RuleID:       private-key
File:         fuite_demo.txt
...
leaks found: 1
```

Un bloc `-----BEGIN PRIVATE KEY-----` déclenche la règle `private-key` de façon fiable. Tu peux aussi tester un faux token GitHub (`ghp_` + 36 caractères) règle `github-pat`.

Nettoie ensuite :

```
git reset HEAD fuite_demo.txt && rm fuite_demo.txt
```

2.2 Bon réflexe : les vrais secrets vont dans `.env`

`.env` est **déjà** dans `.gitignore` (et `.env.example` documente les variables sans valeurs). On ne commite **jamais** `.env`.

2.3 Cas pratiques

- **Faux positif** (ex. un hash de test) : crée `.gitleaks.toml` avec une `allowlist` (regex ou chemin) plutôt que de désactiver gitleaks.
- **Scanner tout l'historique** (au-delà du commit courant) : `gitleaks detect --source . -v`.
- **Contourner en urgence** : `git commit --no-verify` saute *tous* les hooks. À éviter — c'est exactement le filet qu'on vient de poser.

3. Installer DVC + MLflow

3.1 Ajouter les dépendances

On les met dans un groupe optionnel `mlops` pour garder l'install de base légère :

```
uv add --optional mlops "dvc>=3.5" "mlflow>=2.14"
uv sync --extra mlops
```

Bloc obtenu dans `pyproject.toml` :

```
[project.optional-dependencies]
mlops = ["dvc>=3.5", "mlflow>=2.14"]
```

Plus simple mais moins propre : `uv add dvc mlflow` (les met dans les deps principales).

3.2 DVC : versionner les fichiers lourds

Le problème : git gère mal les gros binaires (CSV de données, `.joblib`). **DVC** garde dans git un petit **pointeur** `.dvc` (un hash md5 + la taille), et stocke le vrai fichier dans un **remote** (dossier local, S3, Google Drive...).

```
dvc init # crée .dvc/ (à committer)
dvc remote add -d localremote /tmp/dvc-store # remote local pour la démo
```

Sous **Windows**, utilise un chemin local, ex. `dvc remote add -d localremote C:\dvc-store` (ou un lecteur réseau partagé).

Important : dans ce repo, `data/gold/gold_dataset.csv` et `rf.joblib` sont **déjà suivis par git**. DVC refuse de prendre un fichier suivi par git tant qu'on ne l'a pas sorti de l'index :

```
git rm -r --cached data/gold/gold_dataset.csv artifacts/models/rf.joblib
```

(le script `demo_versioning.py` fait cette migration automatiquement — §5.)

3.3 MLflow : tracer les expériences + registre de modèles

MLflow journalise chaque entraînement (paramètres, métriques, artefacts) et garde un **registre** de versions du modèle. On utilise un backend **SQLite** (un seul fichier `mlflow.db`) — nécessaire pour que le **registre** fonctionne :

```
uv run mlflow ui --backend-store-uri sqlite:///mlflow.db # http://localhost:5000
```

Avec un simple dossier `./mlruns` (sans `--backend-store-uri`), le tracking marche mais le **registre de modèles est indisponible**. D'où le choix de SQLite.

4. Script — flux de données synthétiques (`generate_synthetic_gold.py`)

Ajoute des lignes **synthétiques** au gold dataset, en respectant **exactement** son schéma (15 colonnes) et en **recalculant les features temporelles** (lags/rolling) via la vraie fonction du package aucune fuite, aucun NaN.

```
# Un seul lot de 200 lignes (pour un test / la démo DVC) :
uv run python scripts/generate_synthetic_gold.py --once

# Flux continu : 200 lignes/minute pendant 5 minutes :
uv run python scripts/generate_synthetic_gold.py --minutes 5

# Simuler une dérive capteur (+0,4 °C de moyenne par minute) :
uv run python scripts/generate_synthetic_gold.py --minutes 10 --drift 0.4
```

Option	Défaut	Effet
--once	–	un seul lot puis sortie
--rows-per-minute N	200	lignes ajoutées par tick
--interval S	60	secondes entre deux ticks
--minutes M	∞	durée totale
--ticks N	–	nombre de ticks (prioritaire sur --minutes)
--drift D	0	+D °C ajoutés à la moyenne par tick (dérive)
--freq	1h	pas de temps entre 2 mesures d'une machine
--seed N	–	reproductible

Sémantique : « 200/minute » est le **débit d'ingestion** (temps réel mural). Chaque nouvelle ligne avance l'horloge **de sa machine** de --freq (1 h par défaut), pour rester cohérent avec la série existante. Les 200 lignes sont réparties sur les 4 machines (50 chacune).

Le label panne est **synthétique** (loi logistique corrélée à la température/pression, calibrée ~10,5 %). Ce n'est pas une jointure d'incidents : c'est un flux de **démonstration** pour le versioning.

5. Script — démo versioning (demo_versioning.py)

Un seul script « tout-en-un » qui :

1. **entraîne** le modèle sur le gold (split **temporel par machine**, sans fuite) ;
2. **journalise** params + métriques + modèle dans **MLflow** et **l'enregistre au registre** (indusense-rf) ;
3. **sauvegarde** artifacts/models/rf.joblib + model_metadata.json ;
4. écrit metrics.json / params.yaml (lisibles par `dvc metrics show`) ;
5. **versionne avec DVC** (`dvc add data` + modèle, après migration `gitdvc auto`) ;
6. affiche les commandes **git/dvc** à copier-coller pour figer la version.

```
uv run python scripts/demo_versioning.py --remote /tmp/dvc-store
```

Option	Défaut	Effet
--split	temporal	temporal = honnête · stratified = démo de la fuite (§6.3)
--threshold	auto	seuil de décision (défaut : max-F1 calibré sur le train)
--remote CHEMIN	–	configure un remote DVC local
--no-dvc / --no-mlflow	–	sauter une brique
--n-estimators / --seed / --test-size	200 / 42 / 0.2	hyperparamètres

Sortie (extrait réel) :

```

== 1. Entraînement ==
    split=temporel/machine | train=1516 test=380 | PR-AUC=0.1125 ROC-AUC=0.2365 seuil=0.600 recall=0.0
== 2. MLflow (params + metriques + modele + registre) ==
    modele enregistre au registre : 'indusense-rf'
    run_id=... | tracking=sqlite:///mlflow.db
== 3. DVC (versionner data + modele) ==
    [migration git->dvc] data/gold/gold_dataset.csv etait suivi par git
    -> 2 pointeurs crees : data/gold/gold_dataset.csv.dvc + artifacts/models/rf.joblib.dvc

```

ATTENTION — Le score temporel est volontairement modeste (PR-AUC $\approx$ 0,11, recall $\approx$ 0). Ce n'est pas un bug : sur ce mini-jeu, la distribution des pannes **dérive dans le temps** (ex. MACH-03 : 2 % 19 %), donc un modèle entraîné sur le passé généralise mal hors-temps. C'est exactement le **point pédagogique** (validation temporelle, drift) — voir §6.3.

6. Scénario de démo complet (à dérouler en séance)

6.1 Le fil rouge : data + modèle versionnés ensemble

```

# 0. dépendances (une seule fois)
uv add --optional mlops "dvc>=3.5" "mlflow>=2.14" && uv sync --extra mlops

# 1. Version 1 : run MLflow #1 + modèle v1 + pointeurs DVC
uv run python scripts/demo_versioning.py --remote /tmp/dvc-store
git add data/gold/gold_dataset.csv.dvc artifacts/models/rf.joblib.dvc \
    data/gold/.gitignore artifacts/models/.gitignore .gitignore \
    metrics.json params.yaml .dvc/config
git commit -m "data+model v1"
git tag data-v1

# 2. La data CHANGE (le flux ajoute 200 lignes)
uv run python scripts/generate_synthetic_gold.py --once

# 3. Version 2 : run MLflow #2 + modèle v2 (DVC voit le CSV changer)
uv run python scripts/demo_versioning.py --remote /tmp/dvc-store
git add -A && git commit -m "data+model v2" && git tag data-v2

# 4. Comparer les 2 runs côte à côte
uv run mlflow ui --backend-store-uri sqlite:///mlflow.db    # http://localhost:5000

# 5. Revenir à la v1 (CSV ET modèle reviennent ensemble)
git checkout data-v1 && dvc checkout
git checkout -          # retour au présent

# 6. Pousser data + modèle vers le remote
dvc push

```

Ce que les apprenants voient : un commit git léger (quelques Ko de pointeurs .dvc) référence des données/modèles lourds ; `git checkout <tag> && dvc checkout` restaure **l'état exact** data+modèle d'une version **reproductibilité**. Dans MLflow, chaque run porte le `gold_md5` on relie **quelle donnée** a produit **quel modèle** avec **quelles métriques**.

6.2 Lire les métriques versionnées

```

dvc metrics show          # lit metrics.json
dvc params diff           # diff des params.yaml entre versions

```

6.3 Point d'orgue — montrer la fuite de données

Deux runs à comparer dans MLflow, même data, **split différent** :

```

uv run python scripts/demo_versioning.py --split stratified --no-dvc    # ROC-AUC  $\approx$  0.85  (FUIITE)
uv run python scripts/demo_versioning.py --split temporal   --no-dvc    # ROC-AUC  $\approx$  0.24  (honnête)

```

Le run stratifié paraît **bien meilleur**... c'est une **illusion** : features glissantes + même machine à cheval sur train/test le modèle « voit le futur ». Le split temporel par machine donne la vraie performance. **Message clé** : un bon score peut cacher une fuite ; seule une validation temporelle dit la vérité.

7. CI : tout rejouer à chaque push

Ton `.github/workflows/ci.yml` lance déjà **ruff + black + pytest** sur Python 3.13. Pour ajouter le scan secrets côté CI (en plus du pre-commit local), ajoute un job :

```
secrets:
  runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
    with: { fetch-depth: 0 } # historique complet pour gitleaks
  - uses: gitleaks/gitleaks-action@v2
```

Données en CI : ne fais pas `dvc pull` sans configurer les accès au remote (secrets GitHub Actions). Tant que le remote n'est pas branché, le CI reste reproductible via `uv run indusense build-gold` (reconstruit le gold depuis `data/raw`).

Annexe A — Aide-mémoire commandes

```
# Pre-commit
uv run pre-commit install --hook-type pre-commit --hook-type pre-push
uv run pre-commit run --all-files
uv run pre-commit autoupdate

# Secrets
gitleaks detect --source . -v

# DVC
dvc init
dvc remote add -d localremote /tmp/dvc-store
dvc add data/gold/gold_dataset.csv artifacts/models/rf.joblib
dvc push / dvc pull / dvc checkout
dvc metrics show / dvc params diff

# MLflow
uv run mlflow ui --backend-store-uri sqlite:///mlflow.db

# Scripts
uv run python scripts/generate_synthetic_gold.py --once
uv run python scripts/demo_versioning.py --remote /tmp/dvc-store
```

Annexe B — Dépannage

Symptôme	Cause / solution
output ... is already tracked by SCM	fichier suivi par git <code>git rm -r --cached <fichier></code> puis <code>dvc add</code> (le script le fait)
MLflow : «* Model Registry features are not supported»*	tracking en dossier <code>./mlruns</code> lancer avec <code>--backend-store-uri sqlite:///mlflow.db</code>
recall = 0 / ROC-AUC < 0,5 en split temporel	normal sur ce mini-jeu (drift temporel) — c'est le point pédagogique §6.3
pre-commit modifie des fichiers au 1■■■ run	ruff/black ont reformaté git add les fichiers et recommit
uv / dvc introuvable (Windows)	vérifier le PATH ; relancer le terminal après install ; <code>uv run dvc ...</code>

Symptôme	Cause / solution
gitleaks bloque un faux positif	ajouter une allowlist dans <code>.gitleaks.toml</code> (ne pas désactiver le hook)

Module 24 — Sprint 3 CISIA « Industrialisation & déploiement » · InduSense 4.0 · AELION